作业 4：Bézier 曲线

姓名: 刘博文　学号: 09023225

2025 年 11 月 27 日

1　实验概述

Bézier 曲线是一种常用于计算机图形学的参数曲线。在本实验中，我们实现了 de Casteljau 算法来绘制由 4 个控制点表示的 Bézier 曲线。

代码框架提供了两个核心函数：

- **bezier**：使用控制点序列和 `OpenCV::Mat` 对象绘制 Bézier 曲线。迭代参数 t 从 0 到 1，每步调用 `recursive_bezier` 计算曲线上对应点，并绘制在图像上。
- **recursive_bezier**：使用 de Casteljau 算法递归计算曲线上给定 t 的点。

2　算法说明

2.1　De Casteljau 算法

算法步骤如下：

1. 给定控制点序列 $p_0, p_1, ..., p_n$，先将相邻点连接成线段。
2. 对每条线段按比例 $t : (1-t)$ 计算分割点，生成新的控制点序列。
3. 新序列长度减少 1。
4. 如果序列只包含一个点，则返回该点，否则对新序列递归执行上述步骤。

通过对 $t \in [0, 1]$ 的多次采样，可以得到完整 Bézier 曲线的点集。

3　实验步骤

3.1　代码构建与运行

下载实验框架后，在终端执行：

```
mkdir build
cd build
cmake ..
make
./BezierCurve
```

运行程序后，黑色窗口会显示。点击屏幕选择 4 个控制点，程序将自动绘制 Bézier 曲线。

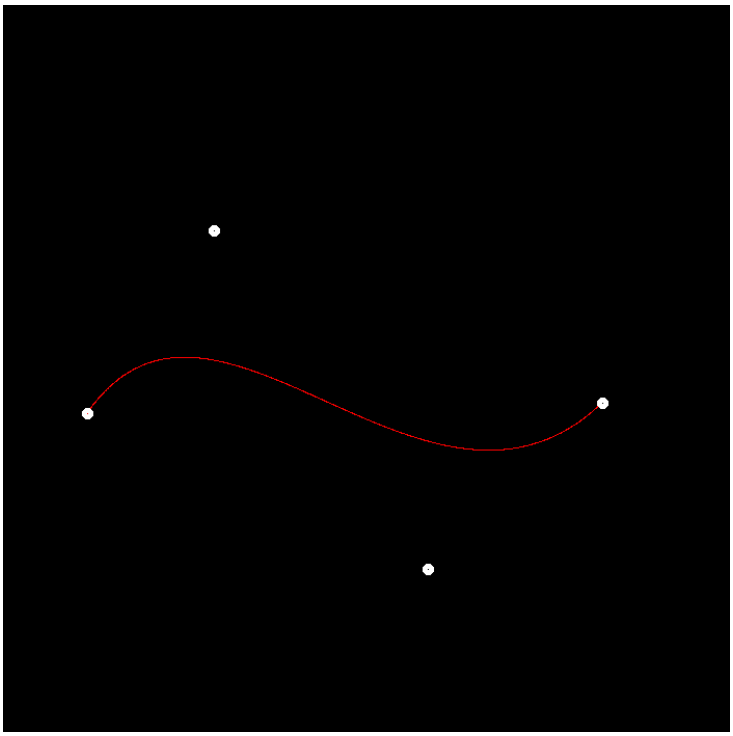

图 1: 使用解析式绘制的 Bézier 曲线（红色）及控制点（白色）

3.2 函数实现说明

recursive_bezier 函数 该函数实现了 **de Casteljau 递归算法**。其核心思想如下：

1. 输入控制点序列 `control_points` 和参数 $t \in [0, 1]$。
2. 若控制点序列长度为 1，则递归终止，返回该点，即曲线上对应的坐标。
3. 否则，依次计算相邻控制点之间按比例 $(1-t) : t$ 的插值点，生成新的控制点序列。
4. 对新的控制点序列递归调用 `recursive_bezier`，直至序列长度为 1。

通过递归不断线性插值，最终得到 Bézier 曲线上对应 t 的精确坐标。具体代码如下：

```cpp
cv::Point2f recursive_bezier(const std::vector<cv::Point2f> &control_points, float t)
{
    // TODO: Implement de Casteljau's algorithm
    // t = std::clamp(t, 0.0f, 1.0f);
    if(t < 0.0f){
        t = 0.0f;
    } else if(t > 1.0f){
        t = 1.0f;
    }
    if(control_points.size() == 1){
        return control_points[0];
    }

    std::vector<cv::Point2f> next;
    next.reserve(control_points.size() - 1);

    for(size_t i = 0; i < control_points.size() - 1; ++i){
        cv::Point2f p = (1.0f - t) * control_points[i] + t * control_points[i + 1];
        next.push_back(p);
    }

    return recursive_bezier(next, t);

    // return cv::Point2f();

}
```

图 2: 补全后的 recursive_bezier 函数

bezier 函数 该函数负责将 Bézier 曲线绘制到 `OpenCV::Mat` 图像上:

1. 设置步长 `step`，迭代 t 从 0 到 1。
2. 每次迭代调用 `recursive_bezier` 获取曲线上的一个点。
3. 将浮点坐标四舍五入，并进行边界检查，避免越界访问图像。
4. 使用 `cv::circle` 绘制曲线点，并用 `cv::line` 连接前后两点以保证曲线连续。

这种方法可以将连续曲线近似为一系列像素点，同时保证曲线平滑且颜色正确。具体代码如下:

```cpp
void bezier(const std::vector<cv::Point2f> &control_points, cv::Mat &window)
{
    // TODO: Iterate through all t = 0 to t = 1 with small steps, and call de Casteljau's
    // recursive Bezier algorithm.
    if(control_points.empty()){
        return;
    }

    const int height = window.rows;
    const int width = window.cols;

    const float step = 0.001f;
    cv::Point prev_pt(-1, -1);

    for(float t = 0.0f; t < 1.0f; t += step){
        cv::Point2f p = recursive_bezier(control_points, t);

        int x = static_cast<int>(std::lround(p.x));
        int y = static_cast<int>(std::lround(p.y));

        if(x >= 0 && x < width && y >= 0 && y < height){
            cv::circle(window, cv::Point(x, y), 1, cv::Scalar(0, 255, 0), -1);
        }

        if(prev_pt.x >= 0){
            if((prev_pt.x >= 0 && prev_pt.x < width && prev_pt.y >= 0 && prev_pt.y < height) &&
               (x >= 0 && x < width && y >= 0 && y < height)){
                cv::line(window, prev_pt, cv::Point(x, y), cv::Scalar(0, 255, 0), 1);
            }
        }
        prev_pt = cv::Point(x, y);
    }
    if(prev_pt.x < 0 || prev_pt.x >= width || prev_pt.y < 0 || prev_pt.y >= height){
        cv::Point2f p1 = recursive_bezier(control_points, 1.0f);
        int x1 = static_cast<int>(std::lround(p1.x));
        int y1 = static_cast<int>(std::lround(p1.y));
        if(x1 >= 0 && x1 < width && y1 >= 0 && y1 < height){
            cv::circle(window, cv::Point(x1, y1), 1, cv::Scalar(0, 255, 0), -1);
        }
    }

}
```

图 3: 补全后的 bezier 函数

如果同时调用 `naive_bezier` 和 `bezier`，结果曲线颜色为黄色，可用于验证算法正确性。

4 实验结果

实验生成的 Bézier 曲线如下：

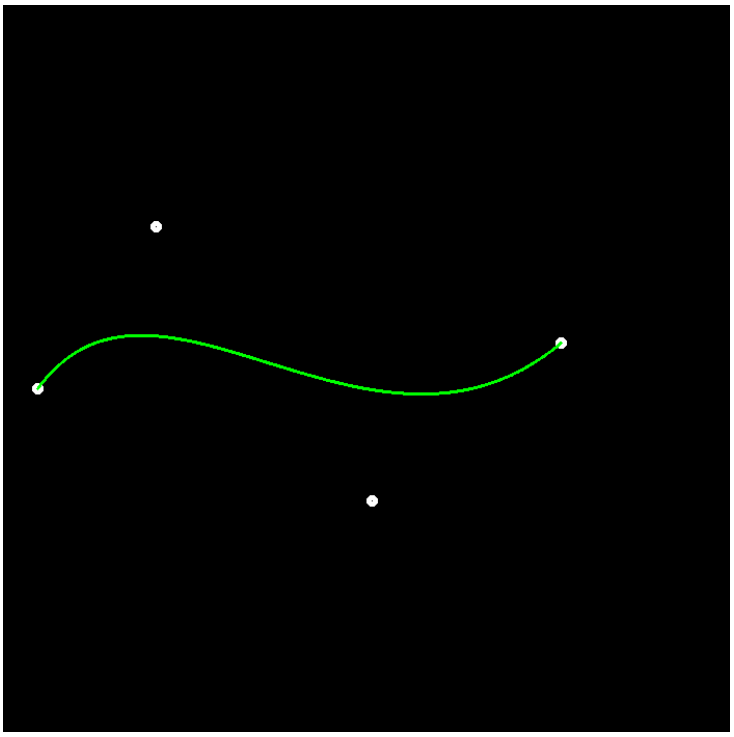

图 4: 使用 de Casteljau 算法绘制的 Bézier 曲线（绿色）及控制点（白色）

同时使用 `naive_bezier` 与 `bezier` 的曲线重叠效果（黄色）如下：

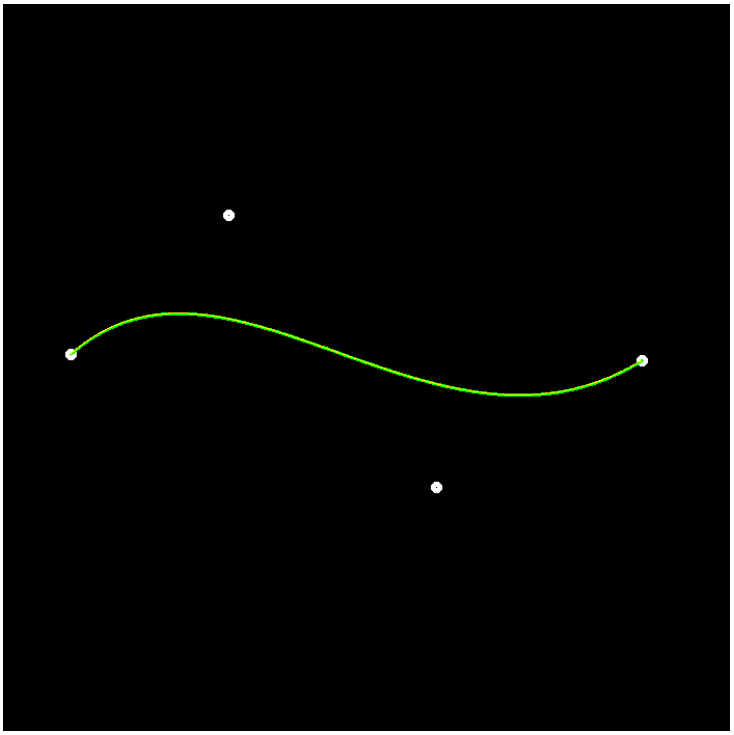

图 5: 两种颜色的曲线叠加效果

5 实验总结

- 本次实验掌握了 de Casteljau 算法的递归实现方法。
- 通过 OpenCV 绘制 Bézier 曲线，理解了参数 t 的作用及曲线生成过程。
- 验证方法：与基于多项式的 `naive_bezier` 对比，结果一致，保证算法正确。
- 可扩展性：修改控制点数量即可绘制更高阶 Bézier 曲线。

6 附录：关键代码片段

6.1 recursive_bezier 函数

```
cv::Point2f recursive_bezier(const std::vector<cv::Point2f> &
   control_points, float t)
{
    if(t < 0.0f){
        t = 0.0f;
    } else if(t > 1.0f){
        t = 1.0f;
    }
    if(control_points.size() == 1){
        return control_points[0];
    }

    std::vector<cv::Point2f> next;
    next.reserve(control_points.size() - 1);

    for(size_t i = 0; i < control_points.size() - 1; ++i){
        cv::Point2f p = (1.0f - t) * control_points[i] + t *
           control_points[i + 1];
        next.push_back(p);
    }

    return recursive_bezier(next, t);
}
```

6.2 bezier 函数

```
void bezier(const std::vector<cv::Point2f> &control_points, cv::Mat &
    window)
{
    if(control_points.empty()){
        return;
    }

    const int height = window.rows;
    const int width = window.cols;

    const float step = 0.001f;
    cv::Point prev_pt(-1, -1);

    for(float t = 0.0f; t < 1.0f; t += step){
        cv::Point2f p = recursive_bezier(control_points, t);

        int x = static_cast<int>(std::lround(p.x));
        int y = static_cast<int>(std::lround(p.y));

        if(x >= 0 && x < width && y >= 0 && y < height){
            cv::circle(window, cv::Point(x, y), 1, cv::Scalar(0, 255,
                0), -1);
        }

        if(prev_pt.x >= 0){
            if((prev_pt.x >= 0 && prev_pt.x < width && prev_pt.y >= 0
                && prev_pt.y < height) &&
               (x >= 0 && x < width && y >= 0 && y < height)){
                cv::line(window, prev_pt, cv::Point(x, y), cv::Scalar
                    (0, 255, 0), 1);
            }
        }
        prev_pt = cv::Point(x, y);
    }
    if(prev_pt.x < 0 || prev_pt.x >= width || prev_pt.y < 0 ||
       prev_pt.y >= height){
        cv::Point2f p1 = recursive_bezier(control_points, 1.0f);
        int x1 = static_cast<int>(std::lround(p1.x));
        int y1 = static_cast<int>(std::lround(p1.y));
        if(x1 >= 0 && x1 < width && y1 >= 0 && y1 < height){
```

```
            cv::circle(window, cv::Point(x1, y1), 1, cv::Scalar(0,
                255, 0), -1);
        }
    }
}
```